

**UNIVERSIDADE FEDERAL DE ALAGOAS  
INSTITUTO DE COMPUTAÇÃO  
BACHARELADO EM CIÊNCIA DA COMPUTAÇÃO**



*DAVI SILVA DE MELO LINS*

**Relatorio Milestone 2 - Projeto: Rede de Relacionamentos Jackut**

POO / JAVA

**Maceió/AL**

2025

*DAVI SILVA DE MELO LINS*

## **Relatorio Milestone 2 - Projeto: Rede de Relacionamentos Jackut**

POO / JAVA

Sistema Jackut - Rede Social

Apresentado no Instituto de Computação da  
Universidade Federal de Alagoas, Campus A. C.  
Simões, como parte dos requisitos para obtenção de  
nota na disciplina de Programação 2 (P2) ou  
Programação Orientada a Objetos (POO).

Orientador: Prof. Dr. Mario Hozano Lucas de Souza

**Maceió/AL**

2025

# SUMÁRIO

## **1. INTRODUÇÃO**

- 1.1 OBJETIVOS
- 1.2 ESCOPO MILESTONE 2
- 1.3 METODOLOGIA

## **2. ESTRUTURA DO PROJETO**

- 2.1 HIERARQUIA DE ARQUIVOS
- 2.2 DIAGRAMA DE CLASSES
- 2.3 PADRÕES DE PROJETO APLICADOS

## **3. DESIGN CLASSES**

- 3.1 Jackute: Coordenação Expandida
- 3.2 GerenciadorUsuarios: Relacionamentos Complexos
- 3.3 GerenciadorComunidades: Gestão de Grupos
- 3.4 Usuario: Novos Atributos e Relacionamentos
- 3.5 Exceções Personalizadas
- 3.6 Persistência e Serialização
- 3.7 Conclusão da Arquitetura

# 1.INTRODUÇÃO

Jackut é uma rede social inspirada em modelos clássicos de relacionamentos online, desenvolvida para demonstrar a aplicação de princípios de design de software e arquitetura modular. Este documento detalha a implementação das User Stories 5 a 9, correspondentes ao **segundo** milestone do projeto, que abrange funcionalidades extras em relacionamentos, comunidades e outros.

## 1.1 Objetivos

O objetivo principal desta etapa foi evoluir a implementação inicial, incorporando:

- Novos tipos de relacionamentos entre usuários
- Sistema de comunidades e mensagens em grupo
- Mecanismo robusto de persistência de dados
- Melhorias na arquitetura com base em feedbacks recebidos

## 1.2 Escopo do Milestone 2

- **US5** – Relacionamentos complexos (ídolos, fãs, paqueras, inimigos)
- **US6** – Gestão de comunidades (criação, associação de membros)
- **US7** – Envio de mensagens em massa para comunidades
- **US8** – Leitura de mensagens e recados com controle de fila
- **US9** – Persistência completa do sistema (estado salvo entre execuções)

A Validação de persistência foi testada em cada UX.2, considerando os arquivos de teste exemplo ( us1\_1.txt) e (us1\_2 txt).

## 1.3 Metodologia

Foi adotada uma abordagem baseada em **Test-Driven Development (TDD)**, utilizando a biblioteca **EasyAccept** para validar o comportamento do sistema frente a cenários de teste pré-definidos.

A arquitetura foi projetada com base em princípios de engenharia de software que favorecem a **separação de responsabilidades**, resultando em uma organização modular do código:

- **Facade:** interface de comunicação com os testes e também entre os serviços.
- **Exceptions:** pacote de exceções personalizadas para controle de erros
- **Models:** entidades principais do sistema, como Usuário e Comunidade
- **Service layers:** Componentes especializados na gestão de usuários, comunidades e relacionamentos, em que também foi utilizado o design pattern de Facade a fim de respeitar a responsabilidade única no projeto.
- **Utilidade:** Classes auxiliares.

## 2.Estrutura do Projeto

### 2.1 Hierarquia de Arquivos

```
├── src/
│   ├── br/
│   │   ├── ufal/
│   │   │   ├── ic/
│   │   │   │   ├── p2/
│   │   │   │   │   ├── jackut/
│   │   │   │   │   │   ├── App/
│   │   │   │   │   │   │   ├── Facade.java
│   │   │   │   │   │   │   ├── Entidades/
│   │   │   │   │   │   │   │   ├── CaixaDeEntrada.java
│   │   │   │   │   │   │   │   ├── Comunidade.java
│   │   │   │   │   │   │   │   ├── Mensagem.java
│   │   │   │   │   │   │   │   ├── Perfil.java
│   │   │   │   │   │   │   │   ├── Recado.java
│   │   │   │   │   │   │   │   └── User.java
│   │   │   │   │   │   └── Exceptions/
│   │   │   │   │   │   │   ├── Comunidade/
│   │   │   │   │   │   │   │   ├── ComunidadeJaExisteException.java
│   │   │   │   │   │   │   │   ├── ComunidadeNaoExisteException.java
│   │   │   │   │   │   │   │   ├── SemMensagensException.java
│   │   │   │   │   │   │   │   └── UsuarioJaNaComunidadeException.java
│   │   │   │   │   │   └── Perfil/
│   │   │   │   │   │   │   ├── AtributoNaoPreenchidoException.java
│   │   │   │   │   │   └── Recado/
│   │   │   │   │   │   │   ├── MensagemParaSiException.java
│   │   │   │   │   │   │   └── SemRecadosException.java
│   │   │   │   │   └── Sistema/
│   │   │   │   │   │   ├── ContaJaExisteException.java
│   │   │   │   │   │   └── LoginOuSenhaInvalidoException.java
│   │   │   │   └── Usuario/
```



- Entidades: Representam os conceitos do domínio;
- Serviços: Implementam a lógica de negócio.

## **2.3 Padrões de Projeto Aplicados**

### **2.3.1 Facade**

- Facade.java e JackutServicesFacade.java implementam o padrão Facade
- Simplificam a interface para operações complexas do sistema
- Centralizam o acesso aos diversos serviços

### **2.3.2 Service Layer**

- Serviços especializados (UserService, CommunityService, etc.)
- Cada serviço gerencia um aspecto específico do sistema
- Promove alta coesão e baixo acoplamento

### **2.3.3 Observer**

- Implementado indiretamente no envio de mensagens para comunidades
- Quando uma mensagem é enviada a uma comunidade, todos os membros são notificados

### **2.3.4 Strategy**

- Validações de relacionamentos no RelationshipService
- Diferentes estratégias para diferentes tipos de relacionamentos (amizade, paquera, etc.)

### **2.3.5 Factory Method**

- PersistenceService atua como fábrica para reconstrução do estado do sistema
- Cria e popula todas as entidades durante a desserialização



### 3.DESIGN DE CLASSES

( O JAVADOC FOI GERADO NA RAIZ DO PROJETO PARA ANÁLISE DE DETALHES ESPECIFICOS )

#### 3.1 JackutServicesFacade: Coordenação Central

A classe JackutServicesFacade foi ampliada para gerenciar todos os aspectos do sistema:

Principais responsabilidades:

- Orquestração entre serviços especializados
- Manutenção da consistência do estado global
- Garantia de integridade nas operações críticas

Métodos-chave:

- adicionarIdolo(), adicionarPaquera(): Gerenciam relacionamentos complexos
- enviarMensagem(): Distribui mensagens para comunidades
- removerUsuario(): Remove usuário e todos seus relacionamentos

#### 3.2 RelationshipService: Gerenciamento de Relacionamentos

Implementa a lógica complexa dos diversos tipos de relacionamentos:

Funcionalidades:

- Amizades com sistema de solicitação/confirmação
- Relações fã/ídolo bidirecionais
- Paqueras com notificação mútua
- Inimizades com bloqueio de interações

Otimizações:

- Uso de estruturas eficientes (HashSet, LinkedHashSet)
- Remoção em cascata de relacionamentos

#### 3.3 CommunityService: Gestão de Comunidades

Novo serviço dedicado à gestão de grupos:

Características:

- Criação e remoção de comunidades
- Adição/remoção de membros
- Envio de mensagens para todos os membros

Estruturas de dados:

- HashMap para acesso rápido por nome
- LinkedHashSet para manter ordem dos membros

#### 3.4 User: Modelagem Expandida

A classe User foi enriquecida com novos atributos e comportamentos:

Novos campos:

- Listas de fãs, ídolos, paqueras e inimigos

- Caixa de entrada com filas separadas para recados e mensagens
- Comunidades como proprietário e participante

Métodos adicionados:

- `getFasString()`, `getPaquerasString()`: Formatação de listas
- Controle de relacionamentos bidirecionais

### 3.5 Exceções Personalizadas

Sistema robusto de tratamento de erros:

Novas exceções:

- `UsuarioEhInimigoException`: Bloqueia interações com inimigos
- `UsuarioJaTemRelacaoException`: Evita duplicação de relacionamentos
- `ComunidadeJaExisteException`: Impede comunidades duplicadas

Organização:

- Pacotes específicos por domínio (Comunidade, Usuario, etc.)
- Mensagens descritivas para facilitar diagnóstico

### 3.6 Persistência e Serialização

Sistema completo de persistência implementado:

Componentes:

- `PersistenceService`: Gerencia carga e salvamento
- `EscritaDeArquivos/LeituraDeArquivos`: Manipulação de arquivos
- Formato textual para fácil inspeção

Funcionalidades:

- Salva estado completo do sistema
- Restaura todos os relacionamentos e comunidades
- Tolerante a falhas (cria novo estado se arquivo corrompido)

## 4. Conclusão da Arquitetura

As decisões arquiteturais desta etapa refletem um compromisso com a evolução sustentável do sistema. Os principais objetivos foram atingidos:

- **Extensibilidade:** Novas funcionalidades foram adicionadas de forma modular, sem comprometer os componentes existentes.
- **Manutenibilidade:** Código bem organizado e documentado
- **Robustez:** Tratamento abrangente de erros e casos extremos
- **Desempenho:** Estruturas de dados adequadas para cada cenário

As decisões técnicas tomadas, como a separação em serviços especializados e o uso de padrões de projeto consagrados, garantem que o sistema está preparado para evoluções futuras de forma sustentável.

Alguns casos teste não passaram, isso após eu transformar a classe User em single responsibility, mas os casos seguiram o mesmo padrão. Tinha que ser devolvida uma lista, e alguns itens dessa lista estavam com a ordem trocada, porém todos os itens estavam na lista. Isso acarretou em alguns casos teste errados, o que não implica uma deficiência total nos métodos, mas uma deficiência parcial, não afetando o funcionamento da rede social jackut.

Além disso, esse relatório contempla os trechos de código que foram utilizados para a composição da Milestone 2 apenas, apesar de que os erros que ocorreram na Milestone 2, são análogos aos erros que ocorreram na Milestone 3.